Reverse Engineering the Sliding Tiles Game

You'll find a sliding tiles game at https://northridge.dev/sliding-tiles. Your job is to "reverse engineer" it – to carefully observe its behavior and puzzle out how it must work.

1. When the blank is in the bottom-right corner, which letter keys disappear from next-move options? Why?

2. When you press `W`, which way does the blank move? Which way does the numbered tile move?

3. If the top-left title has the coordinates `(0, 0)` and the bottom-right coordinates `(3, 3)`, what is the coordinate of:
    a. the `7` tile once the game is solved?

    b. the `10` tile once the game is solved?

4. Think back to the first question. How do you think the code "knows" that a move is impossible?

5. How does the program "know" that the game is over? When does it check?

6. When you press `R` to reset, the tiles get "scrambled". How do you think the code generates a "scrambled" board?

7. How does the code "know" where the blank is?

8. How do you think the code keeps track of where every tile is?

Sliding Tile Game: Flowchart

Imagine you have the following *blocks* available to create this program:

- **IS PUZZLE SOLVED?**
- **SHOW CURRENT BOARD**
- **ASK PLAYER AND WAIT**
- **IS MOVE VALID?**
- **GET NEW BOARD**
- **SWAP TILES?**
- **GET VALID MOVES**

Create a flowchart showing how you could build the sliding tile game from these building blocks.

- Use arrows to show how you move from one block to the next.
- Arrows can “loop back” to show that a block or sequence of blocks needs to be repeated.
- Use a diamond to indicate a decision and its possible outcomes

Sliding Tiles Game: 2D Lists

Let's build our first piece, the `getNewBoard` function. Add this code to game.py.

```
BLANK = "  "  # Note: This string is two spaces, not one.

def getNewBoard():
    """Return a list of lists that represents a new tile puzzle."""
    return [
        ["1 ", "2 ", "3 ", "4 "],
        ["5 ", "6 ", "7 ", "8 "],
        ["9 ", "10", "11", "12"],
        ["13", "14 ", "15", BLANK],
    ]
```

*Note: Each single-digit number has a space **and then** the closing quotation mark.*

This is a *list of lists* or a *2-dimensional* list. There's an "outer" list that holds four "inner" lists. Each inner list represents a column of the game board. **This list of lists will store the current state of our game.**

Testing

The `BLANK` variable and the `getNewBoard` function definition will stay in your code. To test it out and make sure we understand how it works, **temporarily** add this code. Once you're finished, delete it.

```
board = getNewBoard();
print(board[1][3])
```

Notice that we used *two* indexes. The first index (`[1]`) selects an inner list. The second index (`[3]`) selects an element from the inner list.

Answer these questions:

1. What is printed out when you access `board[1][3]`?
2. What do you expect to be printed if you ask for `board[0][1]`? Update your code and check.
3. What do you expect to be printed if you ask for `board[1][0]`? Update your code and check.
4. What indexes (coordinates) could you request in order to print `"15"`?
5. What indexes (coordinates) could you request in order to print `"11"`?
6. Write the indexes (coordinates) for each corner of the game board:
    a. upper-left:
    b. upper-right:
    c. lower-left:
    d. lower-right:

Sliding Tiles Game: Printing the Board

Now that we have a function that can create our internal representation of the game board, let's figure out how to print it so that it looks like tiles.

The function we need is provided. All we have to do is import it and test it.

Add this code to the **top** of your `game.py` file.

```
from utils import displayBoard
```

Then, below your `getNewBoard` function, **temporarily** add this code:

```
board = getNewBoard()
displayBoard(board)
```

And run the program. Voila! You can delete those temporary lines now.

Questions

1. Take a look at the `displayBoard` function definition in `utils.py`. What do you think would happen if you reversed the order of the elements in the `labels` list?

2. What do you think the `format` method does (on the line before the last)?

3. Draw what `displayBoard` would print if you passed it the following list instead of the regular game board:

```
[
    ["a ", "e ", "i ", "m "],
    ["b ", "f ", "j ", "n "],
    ["c ", "g ", "k ", "o "],
    ["d ", "h ", "l ", "p "],
]
```

Sliding Tiles Game: Finding the BLANK

We will need a way to find where in our game board the BLANK tile is located. If you or I were able to look at a board, we could quickly find the blank. But computers don't see like we do. They can't "take it all in". Instead, they have to look at each value one at a time until they find the blank.

If we only had a flat list, that would be easy enough:

```
board = [
  "3 ", "1 ", "4 ", "5 ", "14", "12", "9 ", BLANK,
  "15", "2 ", "6 ", "7 ", "10", "8 ", "11", "13"
]

for idx in range(len(board)):
  if board[i] == BLANK:
    return i
```

But we have a **list of lists**. So we will need a **nested loop** (a loop inside a loop). The first ("outer") loop takes one list (row) at a time. The second ("inner") loop takes one value from at a time for the current list (row).

Add this code to your game.py file (below getNewBoard):

```
def findBlankSpace(board):
    """Return a (row, col) tuple of the blank space's location."""
    for row in range(4):
        for col in range(4):
            if board[row][col] == BLANK:
                return (row, col)
```

Testing

We don't (yet) have a good way to move the blank to a new space. But if you **temporarily** add this code, it should print (3, 3).

```
board = getNewBoard()
location = findBlankSpace(board)
print(location)
```

Question

1. How many values would findBlankSpace have to test before finding the BLANK in a board where the BLANK is at (2, 2)?
2. What would findBlankSpace return if the board didn't have a BLANK space?

Sliding Tiles Game: Making a Move

If we had a physical tile sliding game, we would ***slide*** a tile into the empty space. In code, we will instead ***swap*** the `BLANK` with the appropriate tile. Before we write any code, let's make sure we understand how that works.

Start by writing in each tile its coordinates:

```
+------+------+------+------+
|      |      |      |      |
|  3   |  9   |  2   |  15  |
|      |      |      |      |
+------+------+------+------+
|      |      |      |      |
|  8   |  10  |  14  |  11  |
|      |      |      |      |
+------+------+------+------+
|      |      |      |      |
|  4   |      |  7   |  6   |
|      |      |      |      |
+------+------+------+------+
|      |      |      |      |
|  1   |  12  |  5   |  13  |
|      |      |      |      |
+------+------+------+------+
```

Write the *coordinates* of the two tiles that have to be swapped…

- If the player presses `W` (move up):
- If the player presses `S` (move down):
- If the player presses `A` (move left):
- If the player presses `D` (move right):

Next let's find the pattern. Remember: **The first coordinate gives us the row; the second coordinate gives us the column.**
So `[1][3]` is the **second row** and the **fourth (last) column** (because we start counting from `0`).

	Row Index Changes By	**Column Index Changes By**
`W` (move down)		
`S` (move down)		
`A` (move left)		
`D` (move right)		

Sliding Tiles Game: Swapping Tiles

First, a quick refresher about swapping values in Python. Here's a snippet that will swap the values of `color1` and `color2`:

```
color1 = "gold"
color2 = "maroon"
color1, color2 = color2, color1
```

Now we need to apply that same pattern to the game board. It's more complicated because we're swapping values in a two-dimensional list (a list of lists).

```
+------+------+------+------+
|      |      |      |      |
|  3   |  9   |  2   |  15  |
|      |      |      |      |
+------+------+------+------+
|      |      |      |      |
|  8   |  10  |  14  |  11  |
|      |      |      |      |
+------+------+------+------+
|      |      |      |      |
|  4   |      |  7   |  6   |
|      |      |      |      |
+------+------+------+------+
|      |      |      |      |
|  1   |  12  |  5   |  13  |
|      |      |      |      |
+------+------+------+------+
```

The BLANK is at `[2][1]`. `12` is at `[3][1]`. To move `12` up means to swap them.

```
+------+------+------+------+
|      |      |      |      |
|  3   |  9   |  2   |  15  |
|      |      |      |      |
+------+------+------+------+
|      |      |      |      |
|  8   |  10  |  14  |  11  |
|      |      |      |      |
+------+------+------+------+
|      |      |      |      |
|  4   |  12  |  7   |  6   |
|      |      |      |      |
+------+------+------+------+
|      |      |      |      |
|  1   |      |  5   |  13  |
|      |      |      |      |
+------+------+------+------+
```

The BLANK is now at `[3][1]` and `12` is at `[2][1]`.

Here's code that will perform the swap:

```
board[2][1], board[3][1] = board[3][1], board[2][1]
```

Questions

1. Write code to instead swap `BLANK` and `4`:

2. Write code to instead swap `BLANK` and `7`:

3. Write code to instead swap `BLANK` and `10`:

Generalizing the Pattern

So far, we've performed swaps for a particular spot on the board. Our code only works if `BLANK` happens to be at `[2][1]`.

We would like a *formula* or *function* that can be used no matter where `BLANK` is. If you know the `row` and `col` of the `BLANK` space, you can perform the "up" slide (`W` key) like so:

```
board[row][col], board[row + 1][col] = board[row + 1][col], board[row][col]
```

Let's try to convince ourselves that this works:

BLANK		**Swap Position**	
`row`	`col`	`row`	`col`
0	0		
1	2		
2	3		
3	0		

Questions

In terms of `row` and `col`, what is the *formula* for swapping:

down (`S` key)	
left (`A` key)	
right (`D` key)	

Sliding Tiles Game: `makeMove` Function

Now that we know the formula for swapping tiles, writing a function to execute swaps is pretty simple. It only needs two inputs:

- The current board (`board`)
- The move to execute (`move`, one of `W`, `A`, `S`, `D`)

It uses the `findBlankSpace` function you already wrote to find where the `BLANK` is. The rest of the logic is just four conditionals plus the tiles swap formula you already discovered. Add this function to `game.py`. **Make sure to match the indentation levels.**

```
def makeMove(board, move):
    """Carry out the given move on the given board."""
    row, col = findBlankSpace(board)

    if move == "W":
        board[row][col], board[row + 1][col] = board[row + 1][col], board[row][col]
    elif move == "S":
        board[row][col], board[row - 1][col] = board[row - 1][col], board[row][col]
    elif move == "A":
        board[row][col], board[row][col + 1] = board[row][col + 1], board[row][col]
    elif move == "D":
        board[row][col], board[row][col - 1] = board[row][col - 1], board[row][col]
```

Testing

Let's try each of our moves from the "solved" board. **Temporarily** add this code:

```
board = getNewBoard)
makeMove(board, "S")
makeMove(board, "D")
makeMove(board, "W")
makeMove(board, "A")
displayBoard(board)
```

Draw what the board looks like:

Challenge

Find a sequence of moves (**using each move at least once**) that gets the `BLANK` to the top-left corner. On the back of this sheet, write the sequence and draw what the board looks like after all the moves have been made.

Sliding Tiles Game: Finding Legal Moves

Our `makeMove` function works pretty well, but it *assumes* that the move it accepts as input is “legal”. Think about what will happen if, with a new “solved” board, we try to move left (`A`) or up (`W`). Test it out by **temporarily** adding this code:

```
board = getNewBoard()
makeMove(board, "W")
```

Question: What happens?

To avoid crashing our program, we need a way to ensure that we only accept legal moves. We’ll handle that in the function that asks a user for the next move. Before we write it, let’s find the pattern. Here’s a blank grid. In each, list what moves are impossible (“illegal”). I’ve filled in the top-left corner:

```
+------+------+------+------+
|      |      |      |      |
|   S  |      |      |      |
|   D  |      |      |      |
+------+------+------+------+
|      |      |      |      |
|      |      |      |      |
|      |      |      |      |
+------+------+------+------+
|      |      |      |      |
|      |      |      |      |
|      |      |      |      |
+------+------+------+------+
|      |      |      |      |
|      |      |      |      |
|      |      |      |      |
+------+------+------+------+
```

Now for each possible move, list the coordinates of `BLANK` that make the move impossible:

up (`W` key)	
down (`S` key)	
left (`A` key)	
left (`D` key)	

Sliding Tiles Game: `askForPlayerMove`

Let's write a function that asks a user for a valid move. As we discovered, possible moves are determined by the position of `BLANK`. We we found the pattern. We cannot move:

- up if `BLANK` is on the bottom row
- down if `BLANK` is on the top row
- left if `BLANK` is in the right-most column
- right if `BLANK` is in the left-most column

Our function should:

- Only offer users legal moves
- Only accept legal moves – ask again if the user enters anything else
- Allow the player to quit
- Print out the number of the current move

Here's what that function looks like. Add it to game.py and **be sure to match the spacing and indentation**.

```
def askForPlayerMove(board, num_moves):
    """Let the player select a tile to slide."""
    blank_row, blank_col = findBlankSpace(board)

    w = "W" if blank_row != 3 else " "
    a = "A" if blank_col != 3 else " "
    s = "S" if blank_row != 0 else " "
    d = "D" if blank_col != 0 else " "

    while True:
        print("Move {}".format(num_moves))
        print("                          ({})".format(w))
        print("Enter WASD (or QUIT): ({}) ({}) ({})".format(a, s, d))

        response = input("> ").upper()
        if response == "QUIT":
            return response
        if response in (w + a + s + d).replace(" ", ""):
            return response
```

TURN OVER

Testing

Temporarily add this code to test your function:

```
board = getNewBoard()
move = askPlayerForMove(board, 1)
print(move)
```

Make sure:

- ☐ It only prints valid moves (`W` and `D` since the `BLANK` is in the bottom-right corner)
- ☐ Only accepts valid moves
- ☐ Asks again if you enter something other than a legal move or quit
- ☐ After a valid move is entered, it prints the move
- ☐ Allows the user to quit

Add some moves – for example: `makeMove(board, "D")` – to get the blank to other parts of the board and test again.

- ☐ Top-left corner
- ☐ Top-right corner
- ☐ Bottom-left corner
- ☐ Non-corner edges
- ☐ Center tiles

Sliding Tiles Game: Shuffle the Board

So far we have worked with a “solved” board – a board that already has the tiles properly ordered – or a board that results from a few manual moves. But how do we get the “shuffled” board a user starts from?

You might think you should ***randomize*** the tiles, that maybe you can use the `random` modules we have used in other programs to place tiles randomly. *But that won’t work*. Not every random positioning of the tiles creates a solvable puzzle. So we need another strategy. Let’s work backwards. If we can take a solve board and make 100 or 200 random moves – up, down, left, and right – the board *must* be solvable.

Let’s make a function that can randomly choose a move. That move still has to be legal. We can also pass to the function the last move that was made so that our random move doesn’t just undo the previous move (though the function will still work if we don’t provide the last move).

Add this import to the top of `game.py`:

```
import random
```

And add this function to `game.py`:

```
def makeRandomMove(board, lastMove=None):
    """Perform a slide in a random direction."""
    row, col = findBlankSpace(board)
    validMoves = []
    if row != 3 and lastMove != "S":
        validMoves.append("W")
    if col != 3 and lastMove != "D":
        validMoves.append("A")
    if row != 0 and lastMove != "W":
        validMoves.append("S")
    if col != 0 and lastMove != "A":
        validMoves.append("D")

    move = random.choice(validMoves)
    makeMove(board, move)

    return move
```

Questions

1. List the *two* functions that we created and that are called by `makeRandomMove`.

2. `random.choice(validMoves)` chooses from a list of valid moves. But the `validMoves` list at first is empty. Explain how valid moves get added to it.

Now to get a shuffled board, we just need to make a lot of random moves. Add this function to `game.py`.

```
def getNewPuzzle(moves=100):
    """Get a new puzzle by making random slides from a solved state."""
    board = getNewBoard()

    lastMove = None
    for i in range(moves):
        lastMove = makeRandomMove(board, lastMove)
    return board
```

Testing

Temporarily add this code to test your function:

```
board = getNewPuzzle(20)
displayBoard(board)
```

Did you get a shuffled board? Shuffled enough? Try using different numbers to see what's the smallest number of moves you need to get an adequately shuffled board.

Questions

1. How many times will the variable `lastMove` get updated?

2. What would happen if you updated your code so that you did not pass the last move to `makeRandomMove`?

   ```
   for i in range(moves):
       makeRandomMove(board)
   ```

Sliding Tiles Game: Game Loop Flow Chart

We finally have all the pieces we need. All that remains is to string them together.

Let's make a new flow chart. Your actions are the functions you've written and the function I wrote for you:

- `printInstructions`
- `displayBoard`
- `getNewBoard`
- `getNewPuzzle`
- `askForPlayerMove`
- `findBlankSpace`
- `makeMove`
- `makeRandomMove`

Some of those functions don't need to be called directly; they are only called by other functions. Try to wire the direct functions together so that the result would be a functional game. Anything that's missing will have to be defined in the game loop itself.

Sliding Tiles Game: Game Loop

Here's the `main` function that defines our game loop. Add it to `game.py` and **be careful to match the indentation level**.

```
def main():
    move = 1
    printInstructions()
    input("Press Enter to begin...")

    gameBoard = getNewPuzzle()

    while True:
        displayBoard(gameBoard)
        playerMove = askForPlayerMove(gameBoard, move)
        if playerMove == "QUIT":
            print("Thanks for playing!")
            break
        makeMove(gameBoard, playerMove)
        move += 1

        if gameBoard == getNewBoard():
            print(f"You won in {move} moves!")
            break
```

Then add one last line (**not indented**):

```
main()
```

Testing

Play the game! You should be able to play it until you quit or until you win.

Questions

1. How does this game loop compare to your flow chart? List at least two similarities and at least one difference (unless your flow chart was a *perfect* match).

2. What set up has to happen before the loop starts?

3. How does the game track how many moves the user has played?

4. What are the *two* conditions that can *break* the loop if they evaluate to `True`?